Smart Contract Vulnerability Rectification Engine

Report ID	5de059ed-a97d-4ecc-a525-…
Contract	demo.sol
Language	Solidity
Analysis Date	2026-10-22 16:11:58
Generated	2026-10-22 16:12 UTC
Rectification	Not Fixed

Executive Summary

This report presents the results of an automated security analysis of the smart contract **demo.sol** written in **Solidity**. The analysis was conducted by the SCRE automated pipeline on 2026-10-22. A total of **0 vulnerabilities** were identified. The overall rectification status is **Not Fixed**.

Severity	Count	Assessment
Critical	0	Immediate action required
High	0	Fix before deployment
Medium	0	Review and mitigate
Low	0	Monitor and document
Total	**0**	

Contract Information

Filename	demo.sol
Language	Solidity
Analysis Tool	SCRE v1.0 — Pattern-based detector
Detection Method	Reentrancy pattern (low-level .call before state update)
Repair Strategy	Checks-Effects-Interactions + reentrancy lock
Compilation Check	Solidity 0.8.30 via npx solc

Original Source Code

```
-----------------------------------------------------
--[[
# SOL IS:
* Lua + static typing
* Flexible, plausible typing


# SOL IS NOT:
* Waterproof
* More than Lua (yet!)

--]]



-----------------------------------------------------
-- Undefined globals

--global x = 1337
local  a = x



-----------------------------------------------------
-- Argument checking

local Foo = {}

Foo.member()
Foo:member()
Foo.member(Foo)
Foo:member(Foo)

-- Note - no forward-declare needed!
function Foo:member()
self.member()
self:member()
self.member(self)
self:member(self)
end
```

```
-------------------------------------------------------
-- pairs/ipairs

do
local list = { "one", "two", "three" }
for i,v in pairs(list) do
print(i .. ': ' .. v)
end

local map = { ['one'] = 1, ['two'] = 2, ['three'] = 3 }
for k,v in ipairs(map) do
k = k + '42'
v = v + 42
end
end



-------------------------------------------------------
-- Add a type annotation to the function

local function int_to_string(arg)
return '' .. arg
end

local function string_to_int(arg)
if arg then
return tonumber(arg)
else
return 0
end
end

var i = int_to_string("42")
i = i + 42


local function cmp(a: int?, b: string?) -> bool?
return a == b
--return true
--return nil
end



typedef fmt_string = string -- TODO

local function foo(fmt: fmt_string, ...)
end

foo("hello", 1, 2, 3, 3.14)




-------------------------------------------------------
```

```
-- What is the return type of this?
local function win_or_fail()
if math.random() < 0.5 then
return true
else
return false, "bad luck"
end
end

local win, err_msg, too_many = win_or_fail()




-------------------------------------------------------
-- var vs local

local local_can_be_anything = require 'unfindable'
var   var_must_be_deducible = some_lua_function()
var   var_can_be_explicit   = (require 'unfindable') : int



-------------------------------------------------------
-- Maps and lists:

do
var<[int]> list = {1, 2, 3}
list[1] = 42
list[2] = '1
… [truncated for report]
```

Vulnerability Analysis

No vulnerabilities matching the supported detection patterns were found in this contract.

Suggested Rectifications

Rectification was not applied: No supported repair was applied.

Manual review and remediation by a qualified Solidity developer is recommended.

Security Recommendations

• **Apply Checks-Effects-Interactions (CEI):** Always update contract state before making external calls.

• **Use ReentrancyGuard:** Consider OpenZeppelin's battle-tested ReentrancyGuard modifier for all withdrawal functions.

• **Validate all external call return values:** Never ignore the boolean returned by .call().

• **Implement access control:** Use Ownable or role-based patterns for sensitive functions.

• **Use Solidity ≥0.8.0:** Built-in overflow/underflow protection removes an entire class of arithmetic vulnerabilities.

• **Conduct manual code review:** Automated tools detect known patterns; novel vulnerabilities require expert human review.

• **Deploy to testnet first:** Always test thoroughly on a public testnet before mainnet deployment.

• **Consider a professional security audit:** For any contract handling real funds, engage a reputable auditing firm.

Disclaimer

This report was generated by the SCRE automated analysis prototype. The analysis is pattern-based and limited to a specific subset of known vulnerability classes. A clean result or successful compilation does not guarantee the complete absence of vulnerabilities. Generated repairs must be reviewed, tested, and validated by a qualified Solidity developer before use. SCRE is provided for educational and research purposes only. Do not deploy any contract — repaired or otherwise — to mainnet based solely on this report. The authors accept no liability for damages arising from the use of this tool or its output.